BrainCode Academy

CSV, JSON и проверенный отчёт по поездкам

Неделя 03. Четверг, 90 минут
merge, качество данных, pivot и экспорт

Assylbek Gizatov

Machine learning and Artificial Intelligence

Результат занятия

Объединим поездки со справочником городских зон и сохраним неизвестные ключи.

Проверим, что merge не размножил строки и не потерял сумму стоимости.

Построим отчёт по зонам, сводную таблицу и календарную агрегацию.

Экспортируем CSV и прочитаем его обратно с проверкой структуры и значений.

План на 90 минут

Время	Формат	Содержание
0–18	Теория	CSV/JSON, ключи и типы JOIN
18–45	Практика	Чтение, merge, validate, indicator
45–63	Теория	Качество, сводные, границы метрик
63–90	Практика	Отчёт, pivot, экспорт и разбор

Продолжаем синтетический кейс городских поездок. Все расчёты используют тот же набор, что и во вторник.

Повторение: что должно быть готово

Перед началом: выполняем блок подготовки notebook (DATA_DIR) и код вторника, включая import pandas as pd, raw и trips.

trip_id уникален. requested_at имеет тип datetime и означает время создания заявки.

request_hour, weekday, is_completed и is_cancelled подготовлены. completed содержит только завершённые поездки.

payments_kzt учитывает только completed. Суммы и число строк уже проверены.

CSV и JSON

CSV

Плоская таблица: строка, разделитель, поля.

read_csv(...), to_csv(...).

Типы не закреплены форматом. Разделитель, кодировка и даты требуют внимания.

JSON

Объекты, массивы, числа, строки, null.

json.load(...), DataFrame(...).

Структура может быть вложенной. Наш справочник содержит простой список объектов.

Параметры чтения CSV

```
raw = pd.read_csv(
    DATA_DIR / "trips.csv",
    encoding="utf-8",
    parse_dates=["requested_at"]
)
print(raw.shape, raw["requested_at"].dtype)
```

Для файла с ; используем sep=";". Для десятичной запятой параметр decimal="," задаётся по фактическому формату.

Неверный разделитель может дать один столбец вместо таблицы. Проверка shape и columns обнаружит это сразу.

Преобразование дат и обнаружение ошибок

```
sample = pd.Series(["2026-02-01 08:00", "bad-date"])
parsed = pd.to_datetime(sample, errors="coerce")
invalid = sample.notna() & parsed.isna()
print(invalid.tolist())
```

Результат: [False, True]. errors="coerce" превращает неподходящую дату в NaT, но не исправляет источник.

Считаем такие строки и выясняем причину. Для региональных форматов лучше задавать явный format.

JSON-справочник зон

```
import json
with (DATA_DIR / "zones.json").open(encoding="utf-8") as f:
    records = json.load(f)
zones = pd.DataFrame(records)
print(zones.shape)
zones.head()
```

Одна запись JSON соответствует одной зоне. zone_id связывает справочник с поездками.

json.load читает файл. json.loads читает строку с JSON. Это разные входы.

Справочник zones.json

zone_id	zone_name	zone_type
Z01	Центр	Смешанная
Z02	Университет	Учебная
Z03	Вокзал	Транспортная
Z04	Жилой район	Жилая
Z05	Деловой район	Деловая

Ключ Z99 в справочнике отсутствует намеренно. Названия зон условные и не привязаны к конкретному городу.

Контракт справочника

zone_id должен присутствовать и быть уникальным. Его тип совместим с zone_id в trips.

Название зоны хранится один раз в справочнике. Поездки содержат ссылку на него.

Уникальность справа задаёт many-to-one: много поездок могут ссылаться на одну зону.

Пустые ключи проверяем до merge. pandas может сопоставлять пустые ключи друг с другом.

Проверки ключей до merge

```
assert trips["trip_id"].is_unique
assert trips["zone_id"].notna().all()
assert zones["zone_id"].notna().all()
assert zones["zone_id"].is_unique
print(trips["zone_id"].dtype, zones["zone_id"].dtype)
```

Уникальность trip_id не означает уникальность zone_id в таблице поездок. Повторы зоны ожидаемы.

Если ключи пришли как строки и числа, сначала согласуем формат по смыслу, не удаляя значимые ведущие нули.

Как отличаются типы JOIN

how	Какие ключи сохраняет	Основной риск
left	Все строки левой таблицы	Неизвестные значения справа
inner	Только совпавшие ключи	Потеря строк без пары
outer	Ключи обеих таблиц	Строки только из справочника
right	Все строки правой таблицы	Смена единицы наблюдения

В отчёте о всех запросах на поездку выбираем left. Нужна явная причина, чтобы исключать строки без зоны.

Мини-пример LEFT и INNER JOIN

trip_id	zone_id	После LEFT: zone_name
A	1	Centre
B	2	Station
C	999	NaN

Справочник содержит только зоны 1 и 2. LEFT сохраняет A, B и C. INNER оставляет A и B.

Безопасное объединение many-to-one

```
joined = trips.merge(
    zones, on="zone_id", how="left",
    validate="many_to_one", indicator=True
)
assert len(joined) == len(trips)
assert joined["payments_kzt"].sum() == trips["payments_kzt"].sum()
```

validate проверяет уникальность ключа справа. indicator добавляет происхождение строк в _merge.

Обе проверки полезны: одна описывает структуру связи, другая контролирует итог конкретного отчёта.

indicator показывает строки без пары

```
joined["_merge"].value_counts()
unmatched = joined.loc[
    joined["_merge"].eq("left_only"),
    ["trip_id", "zone_id", "status", "payments_kzt"]
]
unmatched
```

Для LEFT JOIN значения обычно both или left_only. right_only появляется, например, при OUTER JOIN.

Нулевое число unmatched является результатом проверки, а не предположением до чтения данных.

Несовпавшая поездка

trip_id	zone_id	status	Оплата, KZT
1093	Z99	completed	2 700

LEFT JOIN: 168 строк, из них 167 both и 1 left_only. INNER JOIN: 167 строк и потеря 2 700 KZT.

Неизвестная зона остаётся в отчёте

Неизвестный ключ может быть следствием нового объекта, опечатки или неполного справочника.

Автоматически приписать ему известную зону нельзя. Сохраняем отдельную группу Неизвестная зона.

В выводе показываем число таких поездок и их вклад в сумму. Затем задаём вопрос владельцу источника.

Неизвестный канал тоже сохраняется

```
joined["channel_report"] = joined["channel"].fillna("UNKNOWN")
by_channel = joined.groupby("channel_report").agg(
    requests=("trip_id", "size"),
    payments_kzt=("payments_kzt", "sum")
)
assert by_channel["requests"].sum() == len(trips)
```

3 заявки имеют неизвестный channel. Все три completed, их сумма оплат составляет 9 050 KZT.

Без dropna=False или явной метки обычная groupby по channel исключит эти записи.

Отдельная группа Неизвестная зона

```
joined["zone_report"] = joined["zone_name"].fillna("Неизвестная зона")
print(joined["zone_report"].value_counts())
assert joined["zone_report"].notna().all()
```

Альтернатива: groupby("zone_name", dropna=False). По умолчанию groupby исключает пропущенные ключи.

Метка Неизвестная зона делает группу явной и переносится в CSV без неоднозначного пустого названия.

Сколько данных потерял бы INNER JOIN

```
inner = trips.merge(zones, on="zone_id", how="inner",
                    validate="many_to_one")
lost_rows = len(trips) - len(inner)
lost_kzt = trips["payments_kzt"].sum() - inner["payments_kzt"].sum()
print(lost_rows, lost_kzt)
```

Если потерянная поездка была cancelled, денежная сумма может совпасть даже при потере строки.

Поэтому сравниваем и количество строк, и сумму метрики. Одна контрольная сумма недостаточна.

Почему JOIN может размножить строки

Если один zone_id встречается справа дважды, каждая поездка этой зоны получит две строки в результате.

При повторах с обеих сторон получается many-to-many: количество комбинаций растёт.

Уникальный trip_id в исходнике не защищает от ошибки справочника. Нужны validate и проверка результата.

Ожидаемая ошибка контракта

```
bad_zones = pd.concat([zones, zones.iloc[[0]]], ignore_index=True)
try:
    trips.merge(bad_zones, on="zone_id", how="left",
                validate="many_to_one")
except pd.errors.MergeError:
    print("Ключ справочника повторяется: merge остановлен")
```

Испорченная копия нужна только для демонстрации. Исходный zones не меняем.

Нельзя просто убрать validate ради «работающего кода»: нужно устранить неоднозначность ключа.

Практика 1: аудит объединения

За 8 минут: выполните LEFT JOIN с validate и indicator. Выведите все поездки без зоны.

Сравните число строк и сумму payments_kzt до и после объединения.

Затем выполните INNER JOIN и посчитайте потери.

Добавьте одну повторную зону в отдельную копию справочника и объясните ожидаемую ошибку.

Качество данных: пять разных вопросов

Схема: все ли обязательные столбцы присутствуют и имеют подходящие типы?

Уникальность: действительно ли trip_id обозначает один запрос?

Диапазоны и категории: допустимы ли значения длительности и статуса?

Полнота и связи: где неизвестны измерения и какие zone_id отсутствуют?

Сохранение данных: совпали ли число строк и суммы после преобразований?

Проверки допустимых значений

```
allowed = {"completed", "cancelled"}
assert trips["status"].isin(allowed).all()
assert completed["duration_min"].dropna().gt(0).all()
assert completed["fare_kzt"].ge(0).all()
assert trips["requested_at"].notna().all()
```

Проверка положительности относится к известным длительностям. Число пропусков проверяем отдельно: у completed их 2.

Набор статусов и допустимые диапазоны задаёт контракт данных. Проверка исключает конкретный класс ошибок, но не доказывает достоверность измерения.

Отчёт по зонам: именованные агрегаты

```
by_zone = joined.groupby("zone_report").agg(
    requests=("trip_id", "size"),
    completed=("is_completed", "sum"),
    cancellations=("is_cancelled", "sum"),
    cancel_rate=("is_cancelled", "mean"),
    payments_kzt=("payments_kzt", "sum")
)
```

Одна строка отчёта соответствует зоне. Неизвестная зона также получает отдельную строку.

Сумма стоимости сама по себе не показывает эффективность: она зависит от числа поездок и их состава.

Средняя стоимость и нулевой знаменатель

```
denominator = by_zone["completed"].replace(0, float("nan"))
by_zone["mean_fare_completed"] = (
    by_zone["payments_kzt"] / denominator
)
assert by_zone["requests"].sum() == len(trips)
```

Средняя стоимость completed использует число completed, а не все запросы.

Если завершённых поездок нет, средняя стоимость неизвестна. Деление на ноль не превращаем в нулевую среднюю.

Отчёт по зонам: число запросов и отмены

Зона	Запросы	Завершено	Отмены	Отмены, %
Вокзал	36	30	6	16.7
Деловой район	22	16	6	27.3
Жилой район	46	39	7	15.2
Неизвестная зона	1	1	0	0.0
Университет	33	27	6	18.2
Центр	30	26	4	13.3

Сумма запросов = 168. Группа «Неизвестная зона» сохраняет поездку 1093.

Отчёт по зонам: сумма и средняя оплата

Зона	Сумма оплат, KZT	Средняя completed, KZT
Вокзал	63 700	2 123.33
Деловой район	31 550	1 971.88
Жилой район	84 800	2 174.36
Неизвестная зона	2 700	2 700.00
Университет	60 900	2 255.56
Центр	64 750	2 490.38

Контрольная сумма: 308 400 KZT. Средняя в каждой строке делит сумму только на completed.

Сравнение зон без ложных выводов

В отчёте рядом с долей отмен показываем и число запросов, и число отмен.

Большая денежная сумма может означать больше поездок, более длинные маршруты или другой состав запросов.

Наш анализ описательный. Он не устанавливает причину отмен и не является рейтингом качества районов.

Синтетический набор подходит для проверки кода, но не для решений о реальном городе.

pivot и pivot_table

pivot

Перестраивает значения без агрегирования.

Каждой паре index × columns должно соответствовать одно значение.

Повтор пары вызывает ошибку.

pivot_table

Агрегирует повторяющиеся пары.

aggfunc задаёт сумму, среднее или другую функцию.

Значение по умолчанию: mean. Для денежной суммы пишем aggfunc="sum" явно.

Сводная: зона × час

```
zone_hour = joined.pivot_table(
    index="zone_report", columns="request_hour",
    values="payments_kzt", aggfunc="sum", fill_value=0
)
assert zone_hour.to_numpy().sum() == trips["payments_kzt"].sum()
zone_hour
```

Строка задаёт зону, столбец задаёт час. Ячейка содержит сумму стоимости completed.

fill_value=0 относится к пустой агрегированной ячейке по выбранной метрике. Это не правило для duration_min.

Та же сводная через groupby и unstack

```
equivalent = joined.groupby(["zone_report", "request_hour"])[
    "payments_kzt"
].sum().unstack(fill_value=0)
pd.testing.assert_frame_equal(zone_hour, equivalent,
                              check_dtype=False)
```

Два ключа groupby дают многоуровневый индекс. unstack переносит один уровень в столбцы.

Смысл результата тот же. Выбираем форму кода, которую проще объяснить и проверить.

Распределение статусов через crosstab

```
status_counts = pd.crosstab(joined["zone_report"], joined["status"])
status_shares = pd.crosstab(
    joined["zone_report"], joined["status"], normalize="index"
)
status_counts
```

crosstab удобно считает частоты категорий. normalize="index" превращает каждую строку в распределение с суммой 1.

Для интерпретации долей сохраняем рядом исходные частоты. Небольшая группа может давать большую случайную долю.

Сводная по зонам и статусам

Зона	cancelled	completed
Вокзал	6	30
Деловой район	6	16
Жилой район	7	39
Неизвестная зона	0	1
Университет	6	27
Центр	4	26

Значения здесь: количество заявок. Две колонки дают 29 отмен и 139 завершений, всего 168.

Дневная агрегация resample

```
daily = trips.set_index("requested_at")["payments_kzt"].resample("D").sum()
assert daily.sum() == trips["payments_kzt"].sum()
daily.head()
```

resample("D") группирует по календарным дням и требует временной индекс.

День без записей можно считать нулевым только при подтверждённой полноте выгрузки. Иначе это может быть потеря данных.

Что говорит временная структура примера

Каждый из 7 дней содержит ровно 24 заявки. В каждый час с 06 до 17 поступает по 14 заявок за неделю.

Это результат фиксированной конструкции датасета. Распределение по времени не отражает реального спроса.

Различаются статусы и суммы оплат. Их можно группировать для тренировки, но нельзя выдавать за транспортный прогноз.

Для реального анализа нужны покрытие периода, часовой пояс, правила сбора и гораздо больше наблюдений.

Сумма по дням недели и число дат

В нашем наборе 7 дат: по одному дню каждого weekday и по 24 заявки на дату. Это специально заданная конструкция.

В другой выгрузке могут быть пять понедельников и четыре вторника. Тогда сумма смешивает спрос и длительность наблюдения.

Число календарных дат берём из полного периода наблюдения, включая дни с нулём запросов, если полнота подтверждена.

Это пример экспозиции: у сравниваемых групп может быть разная длительность наблюдения.

Практика 2: небольшой аналитический отчёт

За 12 минут: постройте by_zone с запросами, completed, долей отмен и суммой стоимости.

Получите pivot zone_report × request_hour. Проверьте сумму и сохранение Неизвестная зона.

Опишите одну зону с максимальным числом запросов и одну с высокой долей отмен. Укажите размеры групп.

Дополнительный вопрос: какую информацию нужно собрать, прежде чем говорить о причинах различий?

Экспорт: договор о формате

У отчёта должны быть понятные имена полей, единицы измерения и явный ключ.

CSV сохраняет значения, но не всю типовую информацию pandas. Даты после чтения нужно распознать заново.

Отчёт сохраняем отдельно от исходных данных. index=False исключает случайный технический индекс.

После записи читаем файл и проверяем, что потребитель получит ожидаемую структуру.

Экспорт в отдельную папку

```
from pathlib import Path
OUT_DIR = Path("outputs")
OUT_DIR.mkdir(exist_ok=True)
export = by_zone.reset_index()
report_path = OUT_DIR / "zone_report.csv"
export.to_csv(report_path, index=False, encoding="utf-8")
restored = pd.read_csv(report_path)
```

reset_index возвращает zone_report в обычный столбец. index=False предотвращает лишний Unnamed: 0.

Для открытия CSV в некоторых версиях Excel может понадобиться utf-8-sig. Формат выбираем под получателя.

Round-trip: структура, ключи и сумма

```
import numpy as np
assert restored.columns.tolist() == export.columns.tolist()
assert restored["zone_report"].tolist() == export["zone_report"].tolist()
assert len(restored) == len(export)
assert np.allclose(restored["payments_kzt"], export["payments_kzt"])
```

Round-trip означает «записать и прочитать обратно». Он проверяет фактический файл, а не намерение автора кода.

Для типов и всех значений можно добавить assert_frame_equal после явного восстановления нужных dtype.

JSON-экспорт списка объектов

```
json_path = OUT_DIR / "zone_report.json"
export.to_json(json_path, orient="records", force_ascii=False,
               indent=2)
restored_json = pd.read_json(json_path, orient="records")
assert len(restored_json) == len(export)
```

orient="records" создаёт массив объектов, по одному на строку. force_ascii=False сохраняет читаемый Unicode.

Для таблиц с датами явно задаём date_format="iso", а при чтении отдельно проверяем восстановленный тип.

Мини-отчёт: четыре обязательных ответа

Сколько строк пришло, сколько полных дублей удалено и как проверен trip_id?

Сколько поездок не нашли зону и где они отражены в агрегатах?

Что именно означает сумма стоимости и для каких статусов она рассчитана?

Какие контрольные суммы сошлись после merge, groupby, pivot и экспорта?

Связь с будущим ML-проектом

Готовые request_hour, weekday и характеристики зоны могут стать признаками модели.

Длительность или факт отмены могут стать целевой переменной. Вероятность отмены будет предсказанием модели. Заранее определяем момент прогноза.

Статус завершения и итоговая стоимость могут быть неизвестны в момент запроса. Использование будущего факта даст утечку данных.

Сегодня строим корректную таблицу и описательный отчёт. Обучение модели относится к следующим модулям.

Самостоятельная работа

Ориентир: 3–4 часа после двух занятий. Сохраните notebook с выполненными ячейками и своими выводами.

Повторите аудит для зон и часов. Добавьте медиану и p90 длительности completed с числом наблюдений.

Постройте сводную и объясните обработку Неизвестная зона, пропусков и нулевых знаменателей.

Экспортируйте один отчёт, выполните round-trip и напишите 3–5 выводов с числами и ограничениями.

Итог четверга

Вы можете загрузить CSV/JSON, проверить ключи и выбрать подходящий JOIN.

validate защищает контракт связи, indicator показывает несовпавшие строки.

pivot_table и resample помогают менять уровень агрегирования без потери смысла метрик.

Экспорт считается завершённым после проверки фактически сохранённого файла.

Справочник: причины типичных ошибок

MergeError: ключ справа не уникален при many_to_one. Сначала проверяем справочник.

KeyError: неверная метка или имя столбца. Проверяем columns и index.

Потеря Неизвестная зона: группировка по пропущенному ключу без dropna=False.

Лишний Unnamed: 0: CSV сохранил технический индекс. Используем index=False.

Источники и дальнейшее чтение

Городской notebook Week_03_City_Trips_Pandas.ipynb и создаваемые им trips.csv/zones.json. Исходный план темы: week_03_pandas_files в репозитории курса.

pandas User Guide: IO tools, Merge/join/concatenate, Reshaping and pivot tables, Time series.

Исходный syllabus: модуль Python для анализа данных. Понедельная структура и тайминг являются методической организацией.

Городской набор синтетический. Он служит учебной демонстрацией и не описывает работу реального транспортного сервиса.